Claude 交互可视化能力的技术机制与可复现实现方案研究

摘要

2026 年 3 月，Anthropic 宣布 Claude 可以在对话中直接生成交互式图表、图解和其它可视化，以帮助解释复杂概念，并随着对话演进动态更新这些视觉对象。 这一能力是在此前 “Imagine with Claude” 实验基础上的扩展，与 Claude 的 artifacts 应用空间、MCP Apps 交互应用等一系列能力构成了统一的“对话内交互 UI”体系。 本报告在公开资料基础上系统梳理该能力的产品形态、技术基底（尤其是 Model Context Protocol 及其 Apps 扩展）、与 artifacts / 交互 connectors 的关系，并给出一套可在自有智能体中复现的工程级实现方案与架构建议。[1][2] [3][4]

一、产品能力概览：Claude 的对话内交互可视化

1.1 官方特性描述

Anthropic 官方博客指出，Claude 现在可以在对话回复中“内联”生成自定义图表、图解以及其他可视化，并且这些可视化会随着后续对话而被调整或替换。 与侧边栏中的 artifacts 不同，这些可视化是临时的，只为当前解释服务，并在对话演化过程中动态变化或消失。 官方示例包括：说明复利时自动给出一条可交互曲线，以及构建可以点击元素查看详情的元素周期表可视化等。[3][1]
[5][1]

博客明确区分了两类对象：一类是“永久”的 artifacts，用于生成、保存和分享完整工具或文档；另一类是“临时”的图表、图解和可视化，直接嵌入对话气泡中，强调即时理解与交互性。 该功能默认开启，Claude 会在判断视觉解释更合适时主动生成，也可以通过“画成图解”“把这个随时间变化画出来”等显式指令触发。[1][3]

1.2 与 Imagine with Claude 的承接

Anthropic 在视觉能力博客中提到，此次发布的对话内可视化是对此前 “Imagine with Claude” 实验的一种延伸，将原本在独立体验中实现的“实时构建视觉、无需写代码”的能力直接带入常规对话界面。 第三方视频对 Imagine with Claude 的演示显示，Claude 会生成 HTML 和 CSS 片段，由前端在浏览器中渲染成场景或可视化界面。 媒体报道也将此次可视化能力描述为对 Imagine with Claude 的“扩展”，强调用户“在不看到任何代码的情况下即时生成软件式的交互视觉”。[6][3] [^1]

1.3 与 artifacts 的关系

2025 年发布的 artifacts 被定位为“通过对话创建交互式、可分享的 AI 驱动应用和工具”的空间，用户可以在单独的 artifacts 侧栏中创建、浏览和组织这些应用。 更新说明强调 artifacts 现已支持 MCP 与持久化存储，意味着 artifacts 本身可以调用工具、拥有内部状态，并跨会话持久存在。 相比之下，对话内可视化并不作为 artifacts 保存，而是作为特定回答的一部分存在，其生命周期与当前对话片段绑定，更接近“临时交互组件”。[7][5] [^1]

二、Anthropic 交互生态：Artifacts、MCP Apps 与交互 connectors

2.1 交互 connectors 与 MCP Apps 概览

2026 年 1 月，Anthropic 宣布通过 MCP Apps 提供“交互 connectors”，允许在 Claude 对话中直接打开并操作 Asana、Slack、Figma、Amplitude 等工具，例如在对话中编辑 Asana 时间线、预览并发送 Slack 消息、在 FigJam 中生成流程图等。 官方博客指出，这些交互体验构建在 Model Context Protocol（MCP）之上，MCP Apps 是 MCP 的一个扩展，使任何 MCP server 可以将“带 UI 的交互 connectors”注入支持该扩展的任意 AI 客户端，而不仅仅是 Claude。[8][9] [2][1]

第三方分析进一步说明，交互 connectors 使 Claude 成为统一工作空间，而非单纯聊天机器人：用户可以在同一对话中查看 Asana 看板、Slack 线程、Figma 画布等，并直接对其进行操作。 这些体验均依赖 MCP Apps 提供的 UI 能力，将工具 UI 以嵌入式界面形式映射到对话中。[9][10] [2][11]

2.2 MCP Apps 技术要点

MCP Apps 官方博客将其描述为 Model Context Protocol 的首个正式扩展，使 MCP 工具可以返回“交互式 UI 组件”，在对话中以仪表盘、表单、可视化和多步工作流等形式渲染。 从规范和示例中可以提炼出若干关键技术点：[4][11] [^2]

- UI 传输模型：MCP Apps 复用 MCP 的 JSON-RPC 传输，UI 组件通过 `postMessage` 在沙箱 iframe 与宿主应用之间通信，所有交互消息都是结构化、可审计的 JSON-RPC 调用。[12][4]
- App SDK：`@modelcontextprotocol/ext-apps` 包提供了 `App` 类，示例代码展示了 UI 通过 `app.connect()` 建立连接，监听工具结果并基于结果渲染图表，同时可以从 UI 侧调用 server tools 并将用户操作更新回模型上下文。[^2]
- HTML 为主的 UI 载体：MCP Apps 允许 server 提供富 HTML 界面作为 UI 资源，宿主在对话中以嵌入 iframe 的方式渲染这些界面，实现按钮、表单、图表等完整交互能力。[13][4]
- 安全与治理：UI 内容在受限权限的沙箱 iframe 中运行，可配置 CSP 限制外部资源，所有 UI 发起的工具调用都通过宿主可记录和审计的 JSON-RPC 流程，企业可在服务器侧预审 UI 模板与 URL 资源。[11][4] [12][13]
- 文本回退：规范要求 server 对所有 UI 增强工具提供文本降级路径，即即使宿主不支持 UI 扩展，仍可返回有意义的文本结果，从而保持兼容性。[^4]

这些设计使 MCP Apps 成为在对话系统中嵌入交互式 UI 的通用基础设施，不仅支撑 Claude 的交互 connectors，也为 artifacts 与对话内可视化提供了统一的思想框架。[11][2] [^4]

2.3 第三方对 MCP Apps 的解读

社区文章将 MCP Apps 描述为“让 MCP server 在对话客户端里渲染迷你 Web 应用”的机制，UI 可以包含按钮、表单、图表和实时更新等复杂交互。 另一些分析强调 MCP Apps 的企业价值：通过预声明 `ui://` 形式的 UI 资源、沙箱执行和可审计通信，为审批流、数据可视化和合规工作流提供安全可控的交互界面。[12][13]

技术媒体指出，MCP Apps 使 AI 工具“将响应直接变成交互界面”，并列举了 3D 可视化、交互式地图、PDF 高亮审阅、音乐谱面等多种 UI 示例，所有这些都可以跨不同 AI 客户端复用，而无需为每个

客户端单独写前端代码。 从生态角度看，Claude 是最早完整支持 MCP Apps 的客户端之一，这也解释了其在对话内交互 UI 能力上的领先程度。[2][11]

三、对话内可视化的行为与交互特征

3.1 可视化的生成触发

官方说明指出，可视化功能在所有套餐（包括免费版）中默认开启，Claude 会在判断视觉解释更有帮助时自动生成图表或图解，也允许用户通过指令显式要求生成可视化。 第三方使用指南给出了一些典型 prompt 模板，例如“把这组销售数据做成交互条形图”“为入职流程画一张流程图”“创建展示某概念工作原理的图解”等。[14][3] [5][1]

技术媒体报道强调，可视化会“在 Claude 的回答中弹出”，并且是动态的：后续对话可以要求修改轴范围、切换图表类型、放大某个子集或请求更详细的解释，Claude 会更新或替换原有可视化。 这与传统静态图片式回答明显不同，更接近轻量级交互应用。[3][5]

3.2 临时性与上下文依赖

官方博客特别强调，这些对话内可视化是“临时（temporary）”的，它们的唯一目的就是在当前讨论中帮助理解，不会像 artifacts 那样在侧栏长期保存。 随着对话主题转移或回答结构变化，相关可视化会被替换或移除，体现出与对话上下文的强耦合。[5][1]

从交互模式上看，第三方体验总结中反复出现“Claude 在需要时自动拉出自己的小白板（little whiteboard）”的比喻，强调这些可视化是即时生成、即时消化的辅助工具，而非独立产品界面。 这种设计直接影响到架构：可视化的状态多以“当前回答局部状态”形式存在，而非作为独立持久对象。[^5]

3.3 与外部工具可视化的对比

通过 MCP Apps，Claude 还可以在对话中调用如 Hex、Amplitude 等专业分析工具生成交互图表，这类图表本质上由外部 SaaS 提供，并通过 MCP Apps UI 嵌入到对话中。 与之相比，对话内可视化是由 Claude 自身“从零构建”的视觉对象，不依赖外部分析后端，更偏向于教学和解释而非严肃 BI 场景。[10][9] [1][3] [^5]

媒体和第三方博客普遍将这两类体验都归入“对话内交互视觉”的大范畴：一部分是基于外部工具的仪表盘与 BI 图表，另一部分是 Claude 自身生成的教学型图解、流程图、时间轴等。 二者在 UI 层面高度相似，但数据来源和后端依赖存在明显差异，这为在自有智能体中复现提供了两条技术路径（自渲染 vs. 外部工具）。[9][10] [^5]

四、可推断的底层机制：表示形式与渲染路径

> 本节仅基于公开演示与相关技术生态进行推断，用于指导“如何在自有系统中实现类似能力”，并不声称精确还原 Anthropic 内部实现。

4.1 表示层：从文本到结构化可视化描述

Imagine with Claude 的演示表明，Claude 能够生成 HTML + CSS 片段，这些片段被嵌入页面并在浏览器中进行渲染，构成交互式视觉场景。 MCP Apps 和社区实践则表明，使用“结构化 JSON 规格”来描述图表（如 Vega-Lite）是一种对 LLM 友好的模式：JSON 规格简洁、可验证，易于在不同客户端中统一渲染。[15][16] [17][6]

围绕 LLM 生成可视化的研究和工程实践普遍采用如下模式：[18][16] [^15]

- 前端或中间层定义一套“可视化规格”中间表示（IR），可以直接映射到 Vega-Lite、ECharts 或内部渲染引擎。
- LLM 根据用户自然语言请求与数据 schema 生成该规格 IR，而不是生成具体绘图代码。
- 渲染引擎对 IR 进行 schema 校验、修正（如规范化日期字段）、并据此生成实际图表。

类似 Renderify 这类运行时引擎表明，另一条路径是让 LLM 生成 JSX/TSX 或结构化 UI 计划（RuntimePlan JSON），由运行时在浏览器中即时转译与沙箱执行，实现“由 LLM 直接驱动 UI”的能力。 MCP Apps 则将这种思路规范化为跨客户端的标准接口，统一使用 HTML + JSON-RPC 通信。[19][13] [4][2]

综合这些公开实践，可以得出对话内可视化的一般性技术模式：模型生成结构化的视觉规格（HTML/JSON/JSX 等），宿主应用在安全沙箱中渲染该规格，并通过标准化通道（如 MCP Apps 的 JSON-RPC）实现与模型、工具的双向通信。[15][19] [4][2]

4.2 渲染层：宿主应用中的内联组件

MCP Apps 规范明确了在宿主应用中渲染 UI 的方式：UI 内容作为资源被 server 声明，宿主在对话气泡中以 iframe 或等效容器嵌入该 UI，并通过 JSON-RPC over `postMessage` 实现 UI 与宿主间的双向通信。 UI 可以根据收到的工具结果或模型输出渲染图表，也可以把用户的交互（点击、表单输入等）回传给 server 或模型，形成闭环。[13][4] [12][2]

针对对话内可视化，宿主应用只需将“可视化规格”视为一种特殊的 UI 资源：

- 当模型判定需要可视化时，在其输出中产出相应的规格或触发某个 MCP App 工具。
- 宿主拦截该部分结构化内容，初始化一个可视化组件（或 MCP App iframe），并在对应对话消息中渲染。
- 用户交互事件再次通过 MCP Apps 通道或自定义 API 回传，驱动模型或后端更新图表。

此模式与 Claude 调用 Hex/Amplitude 等工具生成图表时的交互路径高度类似，只是对话内可视化的计算逻辑由 Claude 与宿主本身承担，而非外部 SaaS。[10][1] [9][2]

4.3 状态与生命周期管理

由于对话内可视化是“临时”的，其状态管理更接近“与消息绑定的小型 UI 状态机”，而不需要 artifacts 那样的长期持久化存储。 MCP Apps 规范建议将 UI 状态与 server 状态分离：UI 在浏览器沙箱中管理本地交互状态，而重要业务状态由 server 持有，并通过 JSON-RPC 同步。[1][4] [12][13] [^5]

在对话内可视化场景下，可以采用类似策略：

- UI 组件内部持有当前图表视图状态（比如当前选中的元素、缩放级别），这些状态随消息存在或随组件卸载而销毁。
- 若需要跨多轮对话保持一致的可视化逻辑，则由后端或模型记忆“当前解释上下文”，例如当前公式、参数范围等，再由模型重新生成或更新可视化规格。

Artifacts 更新说明提出 artifacts 支持“持久化存储”，用于保存应用状态和用户配置，适合长期使用的工具；而对话内可视化则更偏向轻量级、一次性状态。 在自有系统设计时，可据此区分“教学型临时可视化”和“产品化长期应用”的状态管理策略。[^1]

五、在自有智能体中复现 Claude 式交互可视化的架构方案

本节基于上述分析和公开工程实践，给出一套工程上可落地的“Claude 式对话内可视化”实现蓝图，分为不依赖 MCP 的最小可行实现与基于 MCP Apps 的标准化实现两条路径。

5.1 关键能力拆解

要在自有智能体中复现类似能力，至少需要实现以下关键模块：

- 可视化需求识别：判断何时应该用图表或图解替代纯文本解释，可以由模型自行决策，也可以仅在用户显式提示时启用。[3][5] [^1]
- 结构化视觉规格生成：模型在输出中产生结构化描述（如 JSON 版图表规格、HTML 片段或 JSX/RuntimePlan），而不是最终像素图。[16][19] [^15]
- 前端渲染引擎：客户端或 Web 宿主负责将规格映射到实际 UI 组件，例如基于 Vega-Lite、ECharts 或自研组件库渲染交互图表。[17][18] [^15]
- 交互事件回传通道：用户在图表上的操作（调整参数、点击元素）需要通过统一通道回传到后端或模型，驱动新的调用或回答更新。[4][13] [^2]
- 安全与沙箱：如果允许模型产生 HTML/JSX 等富 UI 内容，需要通过 iframe 沙箱和严格校验避免 XSS 与权限越界，与 MCP Apps 的安全实践保持一致。[12][11] [^4]

5.2 路径一：基于 MCP Apps 的标准化实现

如果你的智能体体系已经或计划采用 Model Context Protocol，则最自然的做法是直接使用 MCP Apps 扩展，将可视化视为一种标准 MCP UI 资源。

5.2.1 架构角色

根据 MCP Apps 规范和参考实现，可以将系统拆分为三类角色：[13][2] [^4]

- 宿主（Host）：你的智能体前端或聊天客户端，负责与模型通信、呈现对话和嵌入 UI iframe。
- MCP server：实现具体数据访问和可视化逻辑的后端服务，对外暴露 MCP 工具与 UI 资源（`ui://`）。
- App UI：运行在浏览器沙箱中的 UI 前端，通过 `@modelcontextprotocol/ext-apps` 的 `App` 类与宿主进行 JSON-RPC 通信，渲染图表并处理交互。[^2]

在 Claude 生态中，Asana、Hex、Amplitude 等服务就以 MCP server + App UI 的形式出现，并通过 MCP Apps 嵌入到 Claude 对话中。[9][10] [1][2]

5.2.2 数据与调用流程

一个典型的“对话内可视化”调用流程可以参考 MCP Apps 示例：

1. 模型规划工具调用：模型根据用户请求与上下文，决定调用某个带 UI 的 MCP 工具（例如 `show_chart`）。[4][2]
2. 宿主调用 MCP server：宿主将工具调用转发给对应的 MCP server，server 返回结构化数据和/或 UI 资源引用（如 `ui://chart/explorer`）。[13][4]
3. 宿主加载 App UI：宿主在当前消息位置嵌入 iframe 加载 App UI，并通过 `App` SDK 建立 JSON-RPC 连接。[^2]
4. UI 渲染与交互：App UI 根据 `toolresult` 数据渲染图表，用户交互（更改参数、点击数据点）通过 `app.callServerTool` 或自定义方法回传。[13][2]
5. 上下文更新：App UI 可通过 `app.updateModelContext` 将关键交互事件写回模型上下文，使后续对话在语义上感知用户在可视化中的操作。[^2]

该模式的优势在于：UI 与数据逻辑均由 MCP server 控制，宿主只负责对话与 UI 宿主，安全边界清晰，并且天然支持多客户端复用。[11][4] [^2]

5.2.3 在自有智能体中应用

在自有系统中，可以采用以下工程策略：

- 为常见可视化场景（时间序列、分组柱状图、散点矩阵、流程图等）设计一套 MCP 工具与对应 UI 资源，例如 `ui://charts/vega-lite`。[16][15] [^13]
- 使用 Vega-Lite 或类似声明式规格作为内部标准，让 MCP server 接收数据和简化参数，由服务器侧构造规范正确的 Vega-Lite JSON，并交给 App UI 渲染。[17][15] [^16]
- 在 App UI 中使用 `App` SDK 监听工具结果、渲染图表并处理交互，将重要交互行为通过 `updateModelContext` 或二次工具调用同步给模型。[13][2]

这一路径本质上是“构建你自己的 Hex / Amplitude 风格的专用可视化工具”，从而在对话内为各种数据源提供交互图表，与 Claude 在 Hex/Amplitude 集成中的行为保持一致。[10][9] [^1]

5.3 路径二：不依赖 MCP 的轻量实现

如果当前不使用 MCP，也可以采用更轻量的“模型 + 前端渲染器”架构，仍然实现与 Claude 相似的对话内可视化体验。

5.3.1 结构化规格设计

参考业界案例，可以为模型设计一个中立的“可视化规格” Schema，例如：

- 基于 Vega-Lite：模型输出 Vega-Lite JSON（或内部 IR 再编译为 Vega-Lite），前端用官方 renderer 或第三方库渲染。[15][16]
- 基于内部 IR：模型生成更高层的中间表示（图表类型、维度、度量、聚合方式、筛选条件等），由后端/前端翻译为 Vega-Lite、ECharts 或其它库。[17][15]
- 基于 RuntimePlan：类似 Renderify，让模型生成 RuntimePlan JSON 这样的 UI 计划，再由运行时引擎转译为 UI 组件（React/Vue 等）。[^19]

实践表明，相比直接生成绘图库代码（例如 Matplotlib/Plotly），声明式 JSON 规格更易于验证与自动纠错，可以通过 Schema 校验、启发式规则和反馈循环提高可靠性。[16][15] [^17]

5.3.2 模型控制逻辑

模型侧需要学习两类任务：

- 何时生成可视化：通过系统 prompt 或后端规则，约束模型在检测到“解释曲线随时间变化”“结构化表格数据”等需求时，倾向于生成可视化规格，而不是纯文本。[3][5] [^1]
- 如何生成规格：通过少量示例（few-shot）和 JSON Schema 约束，让模型生成符合规范的 IR 或 Vega-Lite 规格，必要时通过自动校验与修复循环提升质量。[15][16] [^17]

在对话协议层，可以将“可视化部分”与文本部分区分开，例如在流式输出中使用特殊标记或单独字段承载规格，以便前端解析并渲染对应组件。

5.3.3 前端渲染与交互

前端负责：

- 监听模型输出中的可视化规格片段，创建对应的可视化组件区域。
- 将规格传给渲染库（Vega-Lite、ECharts 等）并挂接交互事件（点击、选择、缩放）。[18][15]
- 将交互回调封装为对话消息或隐式信号回传后端，由后端决定是否再次调用模型（例如“用户点击了某国家，生成更深入的解释”）。[^18]

Tinybird 的实践表明，可以通过“前端组件 + LLM 生成过滤参数 + 后端数据 API”的组合，让用户以自然语言定义实时可视化，前端根据 LLM 输出的结构化参数决定图表类型并渲染。 这一模式与 Claude 在 Hex/Amplitude 集成中的行为高度相似，只是数据源与可视化引擎由你自建。[^18]

5.4 路径三：混合模式——结合 MCP Apps 与内部可视化

在更高级的体系中，可以同时采用 MCP Apps 与内部可视化规格：

- 对于需要访问外部 SaaS / 企业系统的数据，使用 MCP Apps 集成 Hex/Amplitude/内部 BI 平台，将专业图表嵌入对话。[9][10] [1][2]
- 对于轻量教学型图解（算法流程、体系结构、知识图谱等），直接在模型 + 前端层面实现内部可视化规格与渲染器，无需 MCP server。[5][1] [^15]

这种混合模式在工程上能最大程度复现 Claude 的整体体验：既能像 Hex 那样做真实数据分析，又能像 Imagine with Claude 那样即时画出解释性图解和场景。[6][5] [^2]

六、工程与安全考量

6.1 安全沙箱与代码执行

允许 LLM 生成 HTML/JSX 或脚本意味着潜在的 XSS、权限滥用等风险，因此 MCP Apps 规范将 UI 统一限制在沙箱 iframe 中执行，并建议使用 CSP 限制外部资源、对所有输入做严格验证，并避免在 UI 模板中直接暴露敏感数据。[12][11] [4][13]

在自研可视化系统中，即使不使用 MCP，也应借鉴这些原则：

- 若允许模型生成 HTML/JSX，必须在受限 iframe 中渲染，且禁止任意网络访问和高权限 API 调用。[4][12]
- 更推荐使用声明式 JSON 规格（如 Vega-Lite IR），由受信任的渲染器解释，避免直接执行模型生成的脚本逻辑。[16][17] [^15]

6.2 数据治理与权限控制

MCP Apps 的企业实践强调，在展示真实业务数据（如 CRM、日志、BI 指标）时，需要结合现有权限体系控制 server 能访问的数据范围，并在 UI 层避免泄露敏感信息。 在 Hex、Amplitude 等集成案例中，用户需要显式授权工具访问工作区数据，且部分操作（如 Slack 发送消息）需要用户确认。[11][12] [10][1] [^9]

在自有智能体中，若可视化基于内部数据仓库或实时指标，应：

- 在 server 侧强制实施行列级权限与审计日志。
- 对模型侧提供的数据视图做裁剪，只暴露必要字段。
- 对所有 UI 交互触发的写操作（如修改任务、发通知）要求显式确认。

6.3 可观测性与调试

生成式可视化系统复杂度较高，MCP Apps 社区经验建议：

- 记录 UI <-> 宿主 <-> server 的 JSON-RPC 消息，用于审计与调试。[12][4]
- 为可视化规格设计验证与评分机制，检测空图、错误映射等问题，类似 VegaChat 用“Spec Score”与“Vision Score”评估生成质量。[^16]
- 构建回归测试集，对常见可视化需求建立基准规格，避免模型升级导致可视化退化。[17][16]

七、能力边界与可复制性评估

7.1 Claude 能力中可公开复现的部分

从公开资料看，以下部分能力可以在自有智能体中较为直接地复现：

- 对话中自动或按请求生成交互图表、图解和可视化，并随对话动态更新，作为“临时解释工具”存在。[1][3] [^5]
- 通过 MCP Apps 将外部工具（Asana、Slack、Figma、Hex、Amplitude 等）的 UI 嵌入对话，实现跨系统的交互操作。[9][10] [1][2]
- 使用 declarative JSON 规格（如 Vega-Lite）或 RuntimePlan 风格的 UI 计划，让模型生成规格而非执行代码，再由受信任的渲染器渲染。[19][15] [17][16]
- 借助沙箱 iframe、JSON-RPC、CSP 等机制约束 UI，确保可视化与交互能力在企业环境中的安全与可治理性。[11][4] [12][13]

7.2 难以精确复现的内部实现细节

另一方面，Anthropic 未公开以下内部细节，因此很难“逐行照搬”：

- 模型内部如何学习“何时”触发可视化（训练数据、系统提示设计、奖励信号等）。[3][1]
- 对话内可视化的具体表示格式（是 HTML、某种内部 JSON DSL，还是与 MCP Apps 统一的 UI 资源）。
- Claude 客户端内部如何在消息结构层区分文本与可视化 payload，以及如何在多轮对话中统一管理这些组件的生命周期。

不过，从 Imagine with Claude 展示的 HTML/CSS 输出、MCP Apps 对 HTML UI 的强调、以及业界对声明式规格的广泛采用，可以推断 Claude 所采用的方案与本报告所述架构在原则上高度一致：
以结构化规格为核心，辅以安全的前端渲染与标准化交互通道。[6][19] [15][4] [^2]

八、实施建议与落地路线图

8.1 MVP 阶段（1–2 个月）

- 选定一种声明式图表规格（Vega-Lite 或内部 IR + ECharts）。[15][17] [^16]
- 在前端实现“对话消息中内联渲染图表”的组件，支持根据规格 JSON 渲染交互图表。
- 在模型层通过 few-shot + JSON Schema 约束，实现从简单自然语言需求到图表规格的生成能力。
- 用显式指令触发可视化（例如仅在用户明确要求“画图”“可视化”时启用），降低早期误触发成本。[1][3]

8.2 强化阶段（2–4 个月）

- 扩展到流程图、架构图等非数据驱动图解，可考虑使用基于节点/边的内部 IR 再映射到前端图形库。
- 引入自动规格验证与修复流程，利用 schema 校验与启发式规则修正常见错误，必要时通过反馈循环请求模型自我修正。[15][16]
- 初步引入“模型自动决策是否需要可视化”，例如当检测到用户问“随时间变化”“关系结构”等模式时自动提议可视化。[5][1]

8.3 MCP 集成与生态化（4 个月以上）

- 采用 MCP 作为统一工具协议，在现有系统中部署 MCP server；
- 引入 MCP Apps 扩展，将内部可视化服务、BI 工具和审批系统暴露为带 UI 的 MCP 工具。[4][2] [^13]
- 在宿主客户端中实现 MCP Apps UI 扩展，使对话可以同时承载内部可视化与外部 SaaS 的交互界面。[10][9] [^11]
- 构建统一的日志与监控体系，对 UI 渲染、JSON-RPC 消息和模型调用进行全链路追踪，以便在企业场景中满足审计与合规需求。[12][4]

九、结论

Claude 的“对话内交互可视化”并非孤立特性，而是与 artifacts、Imagine with Claude、MCP Apps 以及外部工具连接等能力共同构成的一套统一交互 UI 体系：模型负责规划与生成结构化规格，宿主通过安全沙箱与标准协议渲染和驱动这些视觉组件。 虽然 Anthropic 并未公开全部内部实现细节，但公开博客、演示视频与 MCP Apps 规范已经足以支撑在自有智能体中构建高度相似的能力，关键在于采用声明式规格、标准化交互通道与安全可治理的宿主架构。[6][19] [3][1] [2][4] [^15]

在实践中，推荐以“声明式可视化规格 + 内联渲染组件”作为起点，逐步演进到“基于 MCP Apps 的生态化交互 UI”，从而既能满足教学解释型场景，也能承载真实业务数据与操作，将智能体从“会说话”升级为“会展示、会协同”的交互中枢。[5][1] [2][4]

References

1. Claude builds interactive visuals right in your conversation | Claude - Ask Claude to explain a concept or analyze your data, and it can respond with interactive charts, di...
2. MCP Apps - Bringing UI Capabilities To MCP Clients - “MCP Apps extends the Model Context Protocol in a way that puts humans at the center. The industry h...
3. Anthropic's Claude gets interactive visuals to enhance learning - Last year, Anthropic PBC released a temporary experience called “Imagine with Claude” that enabled i...
4. MCP Apps: Extending servers with interactive user interfaces - MCP Apps Extension introduces a standardized pattern for declaring UI resources, linking them to too...
5. Claude is Tired of Talking: Testing Claude's New Visual Features - Last Fall Anthropic previewed Imagine with Claude, where they basically handed Claude a sketchbook a...
6. Claude Sonnet 4.5 + IMAGINE with Claude - First Look - YouTube - ... Imagine with Claude”. Chapters 02:14 IconTrash Bin 02:17 Icon ... visuals and interactions ✓ Bui...
7. Turn ideas into interactive AI-powered apps | Claude - Artifacts turn anyone into an app creator—no coding needed. Just tell Claude your idea to instantly ...
8. MCP extension unites Claude with apps like Slack, Canva, and Figma - Users will now be able to interact with apps directly inside the Claude chatbot, letting you draft a...
9. Interactive Tools in Claude: Asana, Slack & Figma Inside Chat - Work faster in Claude with Asana, Slack, and Figma apps. Build timelines, draft messages, and edit d...
10. Anthropic Claude Apps: Complete Guide to Slack, Figma & 9 Tool ... - The integration provides interactive data visualization for product analytics teams. ... By embeddin...
11. MCP Apps, the Model Context Protocol's first official extension, turns ... - The Model Context Protocol just got its first official extension with "MCP Apps." AI tools can now e...
12. MCP Apps Extension: Interactive UI for Enterprise AI Agents - Webrix - The MCP Apps Extension brings interactive UIs to AI agents for approvals, data visualization, and co...
13. MCP Apps: Bringing Interactive UIs to AI Conversations - fka.dev - MCP Apps is an extension to the Model Context Protocol that enables servers to deliver interactive u...

14. Build Interactive Charts with Claude | How to AI posted on the topic - You can now build interactive charts directly in Claude. Here's exactly how: → Open Claude → Paste y...
15. Bringing Visualizations to Life in Multi-Agent Systems With Vega-Lite - Vega-Lite uses a concise JSON specification to declaratively describe charts, allowing agents to gen...
16. VegaChat: A Robust Framework for LLM-Based Chart Generation ... - The core of the system is a chart generation pipeline that generates Vega-Lite specifications from r...
17. How to generate better visualizations with LLMs - LinkedIn - Embedding Plotly in a web app can balloon dependencies; Recharts lacks a formal spec; that often lea...
18. Using LLMs to generate real-time data visualizations - Tinybird - Using LLMs to generate user-defined real-time data visualizations lets users create charts by asking...
19. Renderify: A Runtime Engine for Rendering LLM-Generated UI ... - The LLM generates a structured RuntimePlan directly via JSON Schema constraints. This path offers ma...